R1.04 - Architecture et Système d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Gestion des Processus, États, Arborescence (PID/PPID) et Interruption par Signaux POSIX

Synthèse exécutive : Ce cours traite du concept fondamental de processus sous les systèmes d'exploitation de type Unix/Linux (module R1.04). Il détaille le fonctionnement du multitâche préemptif, la distinction essentielle entre un programme passif sur disque et un processus actif en mémoire RAM, la structure arborescente parent-enfant (PID/PPID) issue du processus initial (systemd/init), ainsi que le mécanisme universel de communication asynchrone : les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme au Processus : Définitions, Attributs et Cycle de Vie

Dans un système informatique moderne, le processeur exécute une multitude de tâches simultanément grâce au multitâche préemptif. Le noyau (kernel) Alloue périodiquement des quantum de temps CPU à chaque processus. Pour créer un nouveau processus, Unix utilise l'appel système fork() qui duplique le processus parent, suivi généralement de exec() qui remplace le code du processus enfant par le nouveau programme à exécuter. Tous les processus forment ainsi un arbre généalogique strict remontant jusqu'au processus racine PID 1 (systemd ou init). Lorsqu'un processus effectue une lecture de fichier ou attend une saisie utilisateur, le noyau bascule son état de 'Élu' à 'Bloqué', libérant instantanément le processeur pour un autre processus à l'état 'Prêt'.

- Programme vs Processus : Un programme est une suite d'instructions statique stockée sur le disque dur ; un processus est une instance dynamique en cours d'exécution occupant de la mémoire RAM et des ressources CPU.
- PID et PPID : Le Process ID (PID) est l'identifiant unique attribué par le noyau à chaque processus. Le Parent Process ID (PPID) indique l'identifiant du processus père qui l'a engendré.
- PCB (Process Control Block) : Bloc de contrôle du noyau conservant le registre d'état, les pointeurs mémoire, la priorité, l'identifiant de l'utilisateur (UID) et les descripteurs de fichiers ouverts.
- États d'un processus : Élu (exécute des instructions sur le processeur), Prêt (attend que l'ordonnanceur lui attribue du temps CPU), Bloqué (attend une opération d'entrée/sortie ou un événement externe).

Cas concret enseignant : *Le professeur a utilisé l'analogie de la cuisine : Le programme est le livre de recettes posé sur une étagère (fichier passif sur disque). Le processus est le cuisinier en train de préparer activement le plat dans la cuisine (exécution en mémoire RAM consommant de l'énergie, de l'espace sur le plan de travail et des ustensiles).*

2. Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme la plus basique de communication inter-processus (IPC). Lorsqu'un signal est adressé à un processus cible, le noyau interrompt le fil d'exécution normal de ce processus pour lui faire exécuter un 'gestionnaire de signal' (signal handler). Le développeur peut choisir d'intercepter un signal (catch) ou de l'ignorer (ignore), sauf pour SIGKILL (signal 9) et SIGSTOP (signal 19) qui sont traités exclusivement par le noyau. En contrôle de gestion de tâches (Job Control), 'Ctrl + Z' envoie le signal SIGSTOP pour suspendre temporairement l'exécution, et la commande 'fg' ou 'bg' envoie SIGCONT pour reprendre l'exécution.

- Signal POSIX : Interruption logicielle envoyée à un processus pour lui notifier un événement ou lui ordonner une action.
- SIGINT (Signal 2) : Interruption déclenchée par la combinaison 'Ctrl + C' dans le terminal ; demande l'arrêt modéré d'une tâche interactive.
- SIGTERM (Signal 15) : Demande d'arrêt propre envoyée par défaut par la commande 'kill'. Permet au processus d'exécuter son gestionnaire de nettoyage (sauvegarde de fichiers, fermeture de connexions socket).
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel géré directement par le noyau. Il détruit immédiatement le processus sans possibilité d'être intercepté ou ignoré par celui-ci.

Cas concret enseignant : *Pour faire retenir la différence vitale entre SIGTERM (15) et SIGKILL (9), le professeur a comparé : SIGTERM est le gardien d'immeuble qui frappe à votre porte pour vous demander poliment de ranger vos affaires et d'évacuer le bâtiment (fermeture propre). SIGKILL est l'intervention des forces de sécurité qui détruisent la porte et vous expulsent sur-le-champ sans vous laisser le temps de fermer votre session.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie une erreur alors que 'kill -9 1234' fonctionne ?
Réponse du professeur : En syntaxe shell Bash, le tiret '-' indique qu'il s'agit du numéro du signal à transmettre. Si vous oubliez le tiret et tapez 'kill 9 1234', la commande 'kill' pense que vous souhaitez envoyer le signal par défaut (SIGTERM 15) à deux processus distincts : le PID 9 et le PID 1234. S'il n'existe pas de processus PID 9, le terminal retourne une erreur d'introuvabilité.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment le détruit-on avec la commande kill ?
Réponse du professeur : Un processus zombie (état Z dans ps) a déjà fini son exécution et libéré sa mémoire, mais conserve une ligne dans la table des processus car son père n'a pas encore lu son code de retour via l'appel système wait(). On ne peut PAS tuer un zombie avec 'kill -9' puisqu'il est déjà mort ! Pour le faire disparaître de la table des processus, il faut soit forcer le père à effectuer son wait(), soit tuer le processus père : le zombie sera alors adopté par le processus PID 1 (systemd) qui nettoiera immédiatement la table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, l'étudiant exécute la séquence suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Expliquez le rôle du symbole '&', détaillez le comportement de la commande kill -15 et indiquez ce que retourne un contrôle ultérieur.

Résolution pas à pas :
Étape 1 : Le symbole '&' en fin de ligne lance la commande 'sleep 600' en arrière-plan (background). Le shell rend immédiatement la main à l'utilisateur et affiche le numéro de tâche ainsi que le PID attribué (ex: [1] 4521). Étape 2 : La commande 'ps -ef | grep sleep' filtre l'arborescence des processus pour localiser la ligne du processus sleep et confirme son PID (4521) ainsi que son PPID (le PID du terminal Bash actuel). Étape 3 : 'kill -15 4521' transmet le signal SIGTERM au processus 4521. Le processus sleep reçoit l'instruction d'interruption propre et met fin à son exécution.

Résultat : Le processus sleep est arrêté proprement par le signal SIGTERM. Un second 'ps -ef | grep sleep' confirme que le processus a complètement disparu de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux ?

R : Le processus racine est 'systemd' (ou 'init' sur les anciens systèmes), son PID est toujours 1.

Q : Quel signal est envoyé au processus en premier plan lorsque vous effectuez la combinaison de touches 'Ctrl + C' ?

R : Le signal SIGINT (Signal numéro 2).

Q : Pourquoi le signal SIGKILL (9) ne peut-il jamais être intercepté par un programme informatique ?

R : Parce que SIGKILL n'est pas transmis au gestionnaire de signal du processus : il est exécuté directement par le noyau qui supprime immédiatement le processus de la mémoire.